VoiceOps 2026 – ZentroMart Voice Support Agent

Anjali Prajapat | LLM & Generative AI Course capstone, C&A Club IIT Guwahati × Gnani.ai

1. What I built

One agent, four steps: **audio** → **Gnani STT** → **agent (LLM + RAG + tools)** → **Gnani TTS** → **spoken reply**. The LLM is provider-agnostic (Anthropic or any OpenAI-compatible endpoint). STT and TTS use only the Gnani API (`gnani-vachana` SDK).

- **Step 1 – prompt-only baseline.** A system prompt makes the model ZentroMart's AI support assistant (introduces itself as an AI, short replies) with no documents or tools. Used only as the comparison point.
- **Step 2 – grounded answers (RAG).** The 21 knowledge-base files are split into section-level chunks that keep their [SECTION-ID], status and effective_from. Retrieval is a word + character n-gram TF-IDF hybrid; superseded sections are down-weighted and document type breaks ties (policy > agent guide > help > FAQ > marketing). Each turn gets top-3 pre-retrieved sections and the model can call `search_kb` for more; every chunk used is saved with the answer. If the documents do not cover a question the agent says so and offers a human.
- **Step 3 – conversation.** Verified order data is kept in conversation state so follow-ups ("how long will that take?") work without re-asking. Tools: `lookup_order`, `create_ticket` (dataset mock tools), plus `lookup_product`, `check_pincode` and a deterministic `date_calc` (business days Mon–Sat minus holidays, today = 20 Oct 2026). Code-level guardrails: at most one verification retry, and the agent only says a ticket exists when the tool returned created=True.
- **Step 4 – voice.** `run.py` takes a folder of audio files, runs Gnani STT, the agent and Gnani TTS, and saves for every file the reply text, the reply audio and the document chunks used. Replies are written for the ear (no markdown, order IDs grouped, amounts spoken as rupees); an empty transcript triggers a polite "could you say it again" recovery reply, and a failure in any file still produces a spoken reply.

2. Step 1 vs Step 2 (my own evaluation)

Scored automatically on the practice set with rule-based checks (must_include / must_not_include, ticket created when the expected decision is escalate, and retrieved section IDs vs gold_evidence). 'Completed' shows how many items finished; free-tier token quotas limited some runs, so partial runs are reported as partial.

Run	Completed	Pass	must_include	Extra
Step 1: prompt-only (questions)	52/52	21.2%	25.0%	-
Step 2: RAG + tools (questions)	52/52	90.4%	90.4%	retrieval recall 72.3%
Step 3: conversations (turns)	2/6	88.9%	100.0%	conv pass 50.0%

My first full RAG run (52/52 questions, before the fixes below) scored **67.3% pass** with **70.2% retrieval recall**. Its failures drove the changes in section 3; the table above holds the latest numbers I could finish.

3. What went wrong and what I changed

- **Empty replies.** The reasoning model spent its whole token budget thinking, so some answers came back empty and the agent said "I'm having trouble". Fix: more output-token headroom plus one automatic retry when a reply is empty.
- **Date arithmetic.** The LLM counted refund days wrongly and gave ranges instead of the latest date. Fix: a deterministic `date_calc` tool and a prompt rule to use the maximum of the policy range and answer with the latest date.
- **Handover timing.** The agent asked for an order ID instead of escalating when the customer's own message already showed an escalation condition (e.g. unrecognised order, damaged item over Rs 10,000). It also created tickets too early on verification turns. Fix: explicit handover rules (ticket in the same turn for clear triggers, none on pure verification turns, repeat-contact trigger).
- **Missed special rules.** Sale-period windows and Assured warranty extensions were missed when the model answered from one source. Fix: pre-retrieval of the top sections every turn, and a rule to combine catalog, policy and order data.
- **Strict string checks.** Answers like "27 Oct" or "three" failed checks expecting "27 October 2026" or "3". Fix: style rules for full month names and digits.

- **Tool-call schema errors.** Optional tool arguments sent as null were rejected by the API. Fix: allow null for optional fields and retry once on a failed tool call.
- **Evaluation cost.** A full run needs hundreds of LLM calls and hit daily free-tier limits. Fix: a resumable evaluator that caches every finished item and a `--ids` option to rerun only failing cases.

4. Limitations and what I would fix next

- Retrieval is lexical (TF-IDF); Hindi/Hinglish queries rely on the LLM writing an English search query. A multilingual embedding model would improve recall.
- Gnani REST STT accepts clips up to 60 seconds; longer audio is not split yet. STT language is set per run (`--stt-lang`), with no automatic language detection.
- Scoring is rule-based, so correct answers phrased differently can be marked wrong; an LLM judge would give a fairer estimate.
- Answers depend on the chosen LLM; a smaller model follows the handover and date rules less reliably.

5. Bonus work

- Resumable, cached evaluation harness with subset reruns; deterministic business-day/date tool; Hinglish and Hindi handling; speech-friendly text normalisation; per-file source-chunk logging in batch runs.
- Rules followed: Gnani only for STT/TTS, synthetic dataset only, no hardcoded answers, answers produced by the agent.